Ex:No: 5A Date:	Smart Parking Slot Occupancy Classification Using K-Nearest Neighbor Algorithm

Problem Description:

In a smart city environment, parking management systems collect sensor data to determine whether parking slots are **Occupied** or **Vacant**. Each parking record includes numerical attributes such as **Distance from entry gate**, **Time of day**, **Vehicle density**, and **Parking duration**.

The objective is to classify parking slot status using the **K-Nearest Neighbor (KNN)** algorithm and visualize the classification results for efficient parking management.

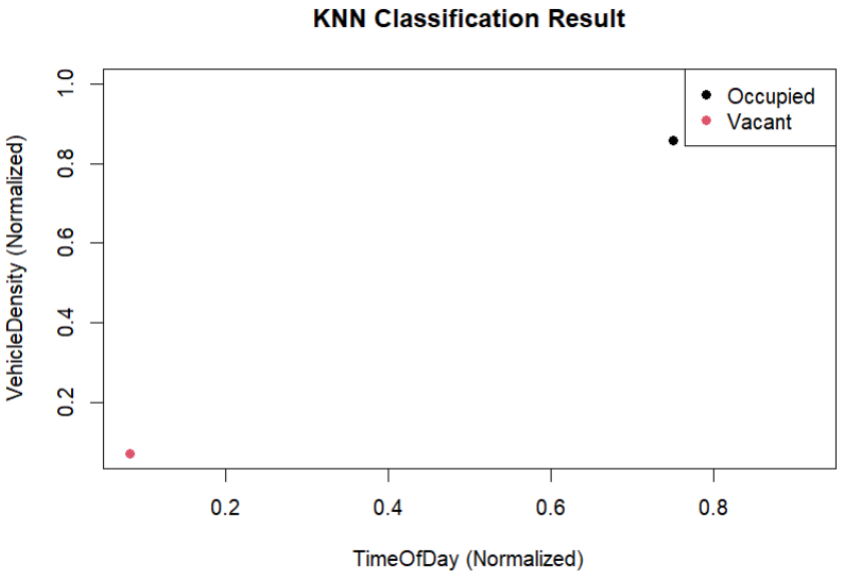
Objective:

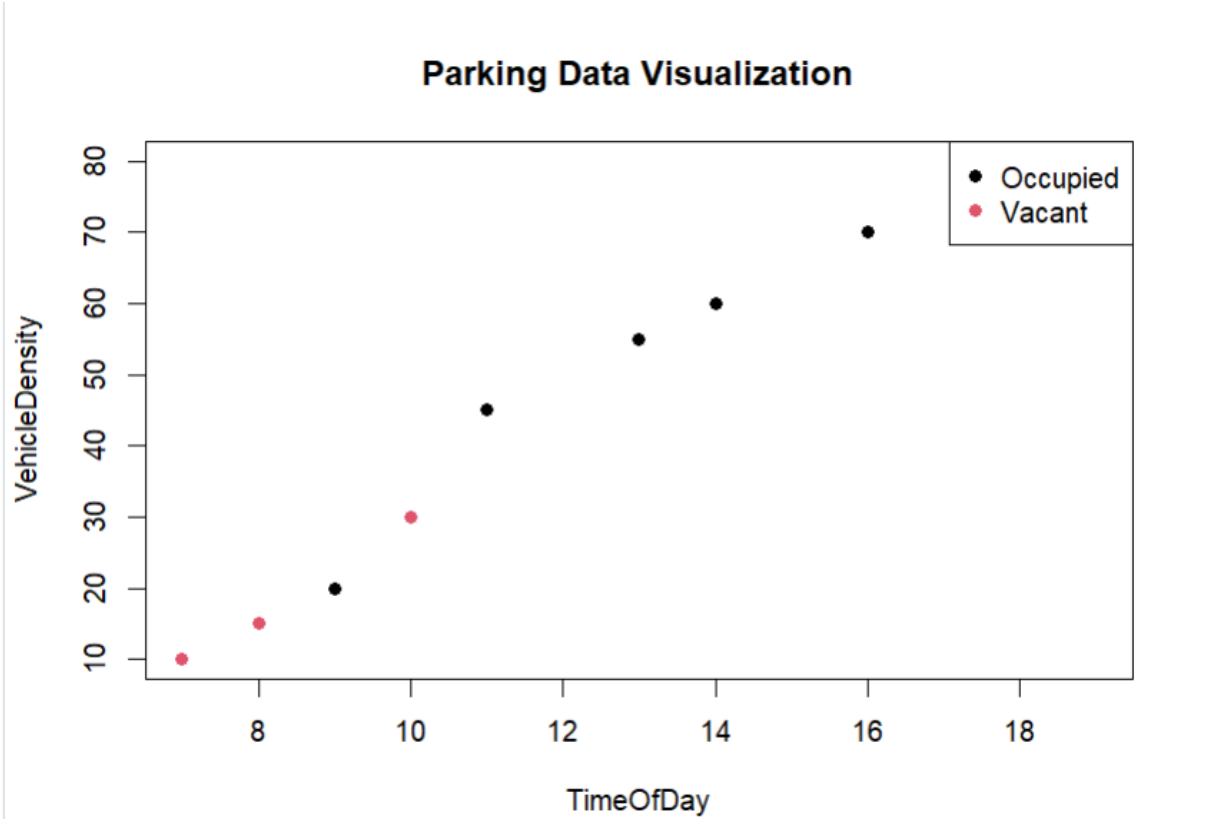
To implement and visualize the K-Nearest Neighbor algorithm in R for classifying parking slot occupancy based on sensor data.

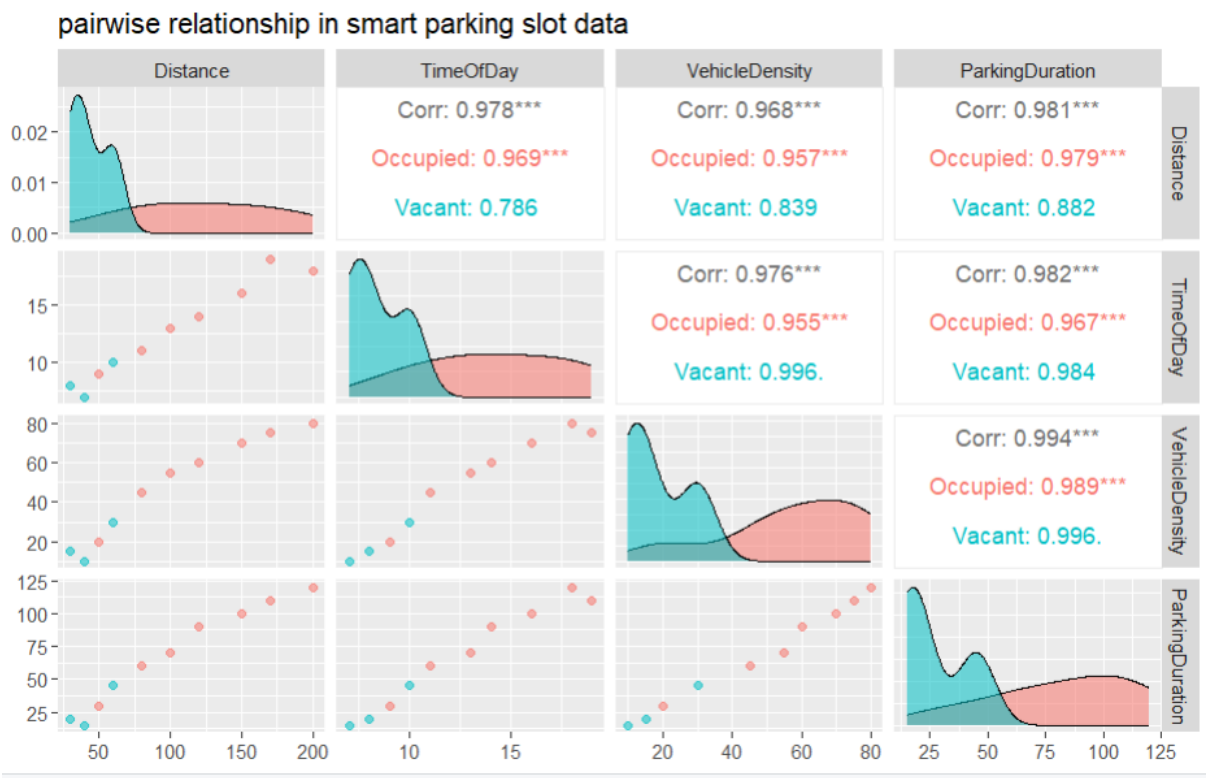
```
# Step 1: Create Parking Dataset
parking_data <- data.frame(
  Distance = c(50,120,80,30,200,60,150,40,100,170),
  TimeOfDay = c(9,14,11,8,18,10,16,7,13,19),
  VehicleDensity = c(20,60,45,15,80,30,70,10,55,75),
  ParkingDuration = c(30,90,60,20,120,45,100,15,70,110),
  Status = c("Occupied","Occupied","Occupied","Vacant","Occupied",
             "Vacant","Occupied","Vacant","Occupied","Occupied")
)

# Step 2: Select Features and Class Label
features <- parking_data[, 1:4]
labels <- parking_data$Status

# Step 3: Data Visualization (Scatter Plot)
plot(parking_data$TimeOfDay, parking_data$VehicleDensity,
     col = as.factor(labels),
     pch = 19,
     xlab = "TimeOfDay",
     ylab = "VehicleDensity",
     main = "Parking Data Visualization")

legend("topright", legend = levels(as.factor(labels)),
       col = 1:length(unique(labels)), pch = 19)

# Step 4: Normalize the Data
normalize <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}
features_norm <- as.data.frame(lapply(features, normalize))

# Step 5: Train-Test Split (70-30)
set.seed(123)
n <- nrow(features_norm)
train_index <- sample(1:nrow(features_norm), 0.7 * nrow(features_norm))

train_data <- features_norm[train_index, ]
test_data <- features_norm[-train_index, ]

train_labels <- labels[train_index]
```

```r
train_labels <- labels[train_index]
test_labels <- labels[-train_index]

# Step 6: Apply KNN (k = 3)
# install.packages("class")
library(class)

predictions <- knn(train = train_data,
                   test = test_data,
                   cl = train_labels,
                   k = 3)

# Step 7: Confusion Matrix and Accuracy
conf_matrix <- table(Predicted = predictions, Actual = test_labels)
cat("\nConfusion Matrix:\n")
print(conf_matrix)

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
cat("\nAccuracy:", accuracy, "\n")

# Step 8: Visualization of Classification Result
plot(test_data$TimeOfDay, test_data$VehicleDensity,
     col = as.factor(predictions),
     pch = 19,
     xlab = "TimeOfDay (Normalized)",
     ylab = "VehicleDensity (Normalized)",
     main = "KNN Classification Result")

legend("topright", legend = levels(as.factor(predictions)),
       col = 1:length(unique(predictions)), pch = 19)

# Step 9: Pairwise Visualization using ggpairs
# install.packages("GGally")
library(GGally)

ggpairs(data.frame(features_norm, Status = labels))
```

```
Confusion Matrix:
           Actual
Predicted  Occupied Vacant
  Occupied        2      0
  Vacant          0      1

Accuracy: 1
```




Parking Data Visualization
Occupied
Vacant
VehicleDensity
TimeOfDay
10
20
30
40
50
60
70
80
8
10
12
14
16
18


pairwise relationship in smart parking slot data
Distance
TimeOfDay
VehicleDensity
ParkingDuration
Corr: 0.978***
Occupied: 0.969***
Vacant: 0.786
Corr: 0.968***
Occupied: 0.957***
Vacant: 0.839
Corr: 0.981***
Occupied: 0.979***
Vacant: 0.882
Corr: 0.976***
Occupied: 0.955***
Vacant: 0.996.
Corr: 0.982***
Occupied: 0.967***
Vacant: 0.984
Corr: 0.994***
Occupied: 0.989***
Vacant: 0.996.

Ex:No: 5B Date:	Visualization and Classification of Iris Flower Species Using K-Nearest Neighbor Algorithm

Problem Description:

The Iris dataset is a well-known dataset in machine learning that contains measurements of iris flowers belonging to three different species: *Setosa*, *Versicolor*, and *Virginica*. Each flower is described using four numerical attributes: **Sepal Length**, **Sepal Width**, **Petal Length**, and **Petal Width**.

The objective is to visualize the dataset, apply the K-Nearest Neighbor (KNN) algorithm, and analyze the classification results.

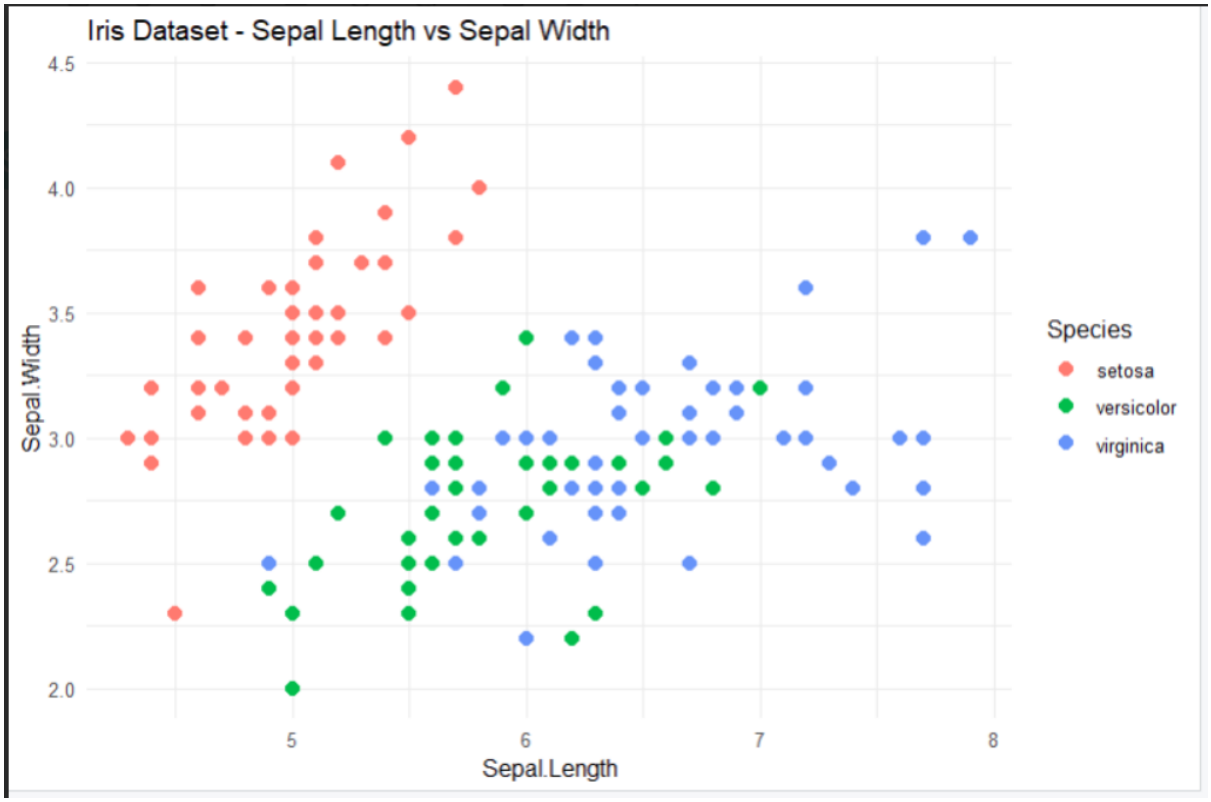
Objective:

To visualize the Iris dataset and implement the K-Nearest Neighbor algorithm in R for classifying iris flower species based on their measurements.

```
1
2  data(iris)
3  set.seed(123)
4  normalize<-function(x){
5    return((x - min(x))/(max(x)-min(x)))
6  }
7  iris_norm<-as.data.frame(lapply(iris[1:4],normalize))
8  iris_norm$Species<-iris$Species
9  library(caret)
10 split_index<-createDataPartition(iris_norm$Species, p=0.8, list = FALSE)
11 train_data<-iris_norm[split_index,]
12 test_data<-iris_norm[-split_index,]
13 train_labels<-train_data$Species
14 test_labels<-test_data$Species
15 library(class)
16 k<-5
17 knn_predictions<-knn(train = train_data[,1:4],
18                       test = test_data[,1:4],
19                       cl = train_labels, k=k)
20 confusion_matrix<-confusionMatrix(knn_predictions, test_labels)
21 print(confusion_matrix)
22 ggplot(iris,aes(x = Sepal.Length, y = Sepal.Width, color = Species))+
23   geom_point(size = 3)+
24   labs(title = "Iris Dataset - Sepal Length vs Sepal Width", x="Sepal Length",
25        y="Sepal Width")+theme_minimal()
26 ggpairs(iris,aes(color = Species))
```

```
> print(confusion_matrix)
Confusion Matrix and Statistics

            Reference
Prediction    setosa versicolor virginica
  setosa          10          0         0
  versicolor       0         10         2
  virginica        0          0         8

Overall Statistics

               Accuracy : 0.9333
                 95% CI : (0.7793, 0.9918)
    No Information Rate : 0.3333
    P-Value [Acc > NIR] : 8.747e-12

                  Kappa : 0.9

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor Class: virginica
Sensitivity                 1.0000            1.0000           0.8000
Specificity                 1.0000            0.9000           1.0000
Pos Pred Value              1.0000            0.8333           1.0000
Neg Pred Value              1.0000            1.0000           0.9091
Prevalence                  0.3333            0.3333           0.3333
Detection Rate              0.3333            0.3333           0.2667
Detection Prevalence        0.3333            0.4000           0.2667
Balanced Accuracy           1.0000            0.9500           0.9000
```


Sepal.Length
Sepal.Width
Petal.Length
Petal.Width
Species
Corr: -0.118
setosa: 0.743***
versicolor: 0.526***
virginica: 0.457***
Corr: 0.872***
setosa: 0.267.
versicolor: 0.754***
virginica: 0.864***
Corr: 0.818***
setosa: 0.278
versicolor: 0.546***
virginica: 0.281*
Corr: -0.428***
setosa: 0.178
versicolor: 0.561***
virginica: 0.401**
Corr: -0.366***
setosa: 0.233
versicolor: 0.664***
virginica: 0.538***
Corr: 0.963***
setosa: 0.332*
versicolor: 0.787***
virginica: 0.322*
setosa
versicolor
virginica


Iris Dataset - Sepal Length vs Sepal Width
Sepal.Width
Sepal.Length
Species
setosa
versicolor
virginica